LABSHEET -10 COMPUTER VISION

```
# 1. IMPORT LIBRARIES
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from tensorflow.keras import models, layers

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

```
# 2. DATASET PATH
dataset_path = "/content/digit-recognizer.zip"
train_file = os.path.join(dataset_path, "train.csv")
test_file = os.path.join(dataset_path, "test.csv")
```

```
# 3. CHECK FILES
print("\n============================================================")
print("CHECKING DATASET")
print("============================================================")
print("PRIYANSHU SHARMA CU240250980")
print("Train file:", train_file)
print("Test file :", test_file)

if not os.path.exists(train_file):
    print("\nERROR: train.csv not found!")
    print("Please check the dataset folder.")
    exit()

if not os.path.exists(test_file):
    print("\nERROR: test.csv not found!")
    print("Please check the dataset folder.")
    exit()

print("\nDataset files found successfully!")
```

```
============================================================
CHECKING DATASET
============================================================
PRIYANSHU SHARMA CU240250980
Train file: /content/digit-recognizer.zip/train.csv
Test file : /content/digit-recognizer.zip/test.csv

ERROR: train.csv not found!
Please check the dataset folder.

ERROR: test.csv not found!
Please check the dataset folder.

Dataset files found successfully!
```

```
# 4. LOAD CSV DATA

import pandas as pd
import zipfile

zip_file = "/content/digit-recognizer.zip"

with zipfile.ZipFile(zip_file, 'r') as z:
    z.extractall("/content/dataset")

train_file = "/content/dataset/train.csv"
test_file = "/content/dataset/test.csv"

train_data = pd.read_csv(train_file)
test_data = pd.read_csv(test_file)
```

```
print("Training data shape:", train_data.shape)
print("Testing data shape :", test_data.shape)
print(train_data.head())

print("PRIYANSHU SHARMA CU240250980")
```

```
Training data shape: (42000, 785)
Testing data shape : (28000, 784)
   label  pixel0  pixel1  pixel2  pixel3  pixel4  pixel5  pixel6  pixel7  \
0      1       0       0       0       0       0       0       0       0
1      0       0       0       0       0       0       0       0       0
2      1       0       0       0       0       0       0       0       0
3      4       0       0       0       0       0       0       0       0
4      0       0       0       0       0       0       0       0       0

   pixel8  ...  pixel774  pixel775  pixel776  pixel777  pixel778  pixel779  \
0       0  ...         0         0         0         0         0         0
1       0  ...         0         0         0         0         0         0
2       0  ...         0         0         0         0         0         0
3       0  ...         0         0         0         0         0         0
4       0  ...         0         0         0         0         0         0

   pixel780  pixel781  pixel782  pixel783
0         0         0         0         0
1         0         0         0         0
2         0         0         0         0
3         0         0         0         0
4         0         0         0         0

[5 rows x 785 columns]
PRIYANSHU SHARMA CU240250980
```

```
# 5. SEPARATE LABEL AND FEATURES
# ------------------------------------------------------------

print("\n============================================================")
print("SEPARATING FEATURES AND LABEL")
print("============================================================")

# Label column contains digit 0-9
y = train_data["label"]

# Remove label column
X = train_data.drop("label", axis=1)

# Test data contains only pixel values
X_test_final = test_data

print("X shape:", X.shape)
print("y shape:", y.shape)
print("Test shape:", X_test_final.shape)
```

```
============================================================
SEPARATING FEATURES AND LABEL
============================================================
X shape: (42000, 784)
y shape: (42000,)
Test shape: (28000, 784)
```

```
# 6. CONVERT DATA TO NUMPY
# ------------------------------------------------------------

X = X.values
y = y.values
X_test_final = X_test_final.values
```

```
# 7. NORMALIZE PIXEL VALUES
# ------------------------------------------------------------

print("\n============================================================")
print("PREPROCESSING DATA")
print("============================================================")
print("PRIYANSHU SHARMA CU240250980")
X = X.astype("float32") / 255.0
X_test_final = X_test_final.astype("float32") / 255.0
```

```
============================================================
PREPROCESSING DATA
```

```
# 8. RESHAPE IMAGES
# ----------------------------------------------------------------------------------------

# 784 pixels = 28 x 28 image

X = X.reshape(-1, 28, 28, 1)

X_test_final = X_test_final.reshape(-1, 28, 28, 1)
print("PRIYANSHU SHARMA CU240250980")
print("Training image shape:", X.shape)
print("Test image shape    :", X_test_final.shape)
print("PRIYANSHU SHARMA CU240250980")
```

```
PRIYANSHU SHARMA CU240250980
Training image shape: (42000, 28, 28, 1)
Test image shape    : (28000, 28, 28, 1)
PRIYANSHU SHARMA CU240250980
```

```
# 9. DISPLAY SAMPLE IMAGES

import matplotlib.pyplot as plt

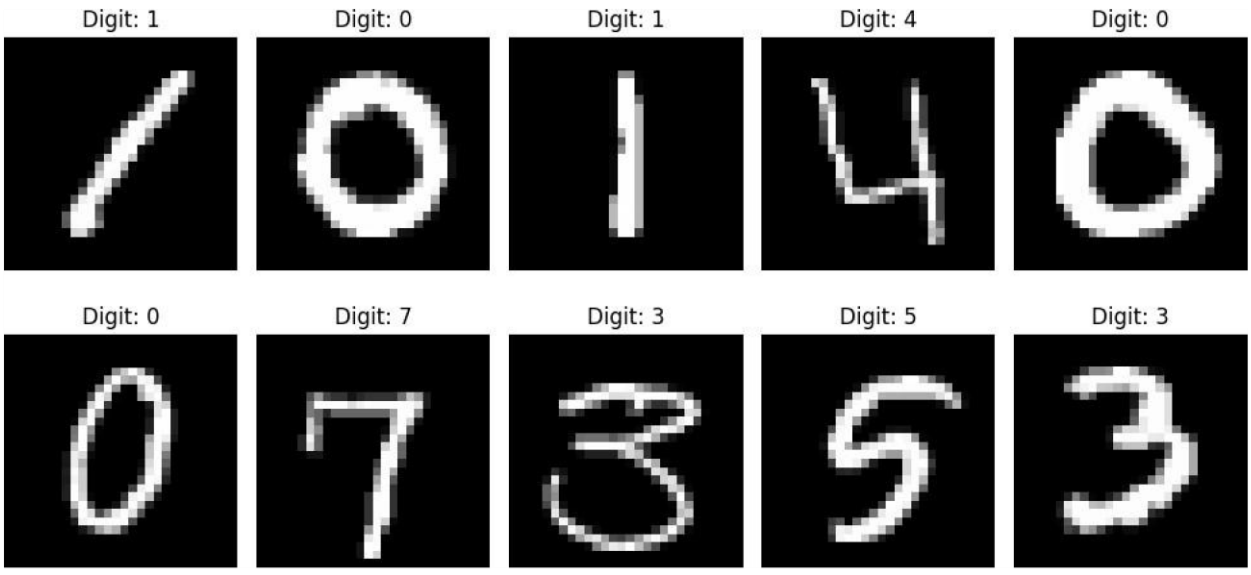
X = train_data.drop("label", axis=1).values
y = train_data["label"].values

plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X[i].reshape(28, 28), cmap="gray")
    plt.title("Digit: " + str(y[i]))
    plt.axis("off")

plt.tight_layout()
plt.show()

print("PRIYANSHU SHARMA CU240250980")
```




```
PRIYANSHU SHARMA CU240250980
```

```
# 10. SPLIT DATA INTO TRAINING AND VALIDATION
# ----------------------------------------------------------------------------------------
from sklearn.model_selection import train_test_split

X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.10, random_state=42, stratify=y
)

print("Training images  :", X_train.shape)
print("Validation images:", X_val.shape)
print("PRIYANSHU SHARMA CU240250980")
```

```
Training images  : (37800, 784)
Validation images: (4200, 784)
```

```
PRIYANSHU SHARMA CU240250980
```

```
# 11. BUILD CNN MODEL

import tensorflow as tf
from tensorflow.keras import models, layers

model = models.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax")
])

model.summary()

print("PRIYANSHU SHARMA CU240250980")
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)
PRIYANSHU SHARMA CU240250980

```
# 13. COMPILE MODEL
# ------------------------------------------------------------

print("\n============================================================")
print("COMPILING MODEL")
print("============================================================")

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully.")
```

```
============================================================
COMPILING MODEL
============================================================
Model compiled successfully.
```

```
# 14. TRAIN MODEL

import numpy as np

# Reshape only if needed
if X_train.ndim == 2:
    X_train = X_train.reshape(-1, 28, 28, 1)

if X_val.ndim == 2:
    X_val = X_val.reshape(-1, 28, 28, 1)

# Normalize
X_train = X_train.astype("float32") / 255.0
X_val = X_val.astype("float32") / 255.0

# Compile model
model.compile(
```

```
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Train
history = model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_data=(X_val, y_val),
    verbose=1
)

print("MODEL TRAINING COMPLETED")
print("PRIYANSHU SHARMA CU240250980")
```

```
Epoch 1/5
591/591 ━━━━━━━━━━━━━━━━━━━━ 30s 48ms/step - accuracy: 0.7028 - loss: 0.9407 - val_accuracy: 0.8781 - val_loss: 0.3878
Epoch 2/5
591/591 ━━━━━━━━━━━━━━━━━━━━ 29s 49ms/step - accuracy: 0.8992 - loss: 0.3304 - val_accuracy: 0.9188 - val_loss: 0.2653
Epoch 3/5
591/591 ━━━━━━━━━━━━━━━━━━━━ 41s 48ms/step - accuracy: 0.9296 - loss: 0.2312 - val_accuracy: 0.9345 - val_loss: 0.2022
Epoch 4/5
591/591 ━━━━━━━━━━━━━━━━━━━━ 47s 58ms/step - accuracy: 0.9490 - loss: 0.1684 - val_accuracy: 0.9507 - val_loss: 0.1592
Epoch 5/5
591/591 ━━━━━━━━━━━━━━━━━━━━ 28s 47ms/step - accuracy: 0.9596 - loss: 0.1319 - val_accuracy: 0.9640 - val_loss: 0.1211
MODEL TRAINING COMPLETED
PRIYANSHU SHARMA CU240250980
```

```
# 15. ACCURACY GRAPH
# ------------------------------------------------------------

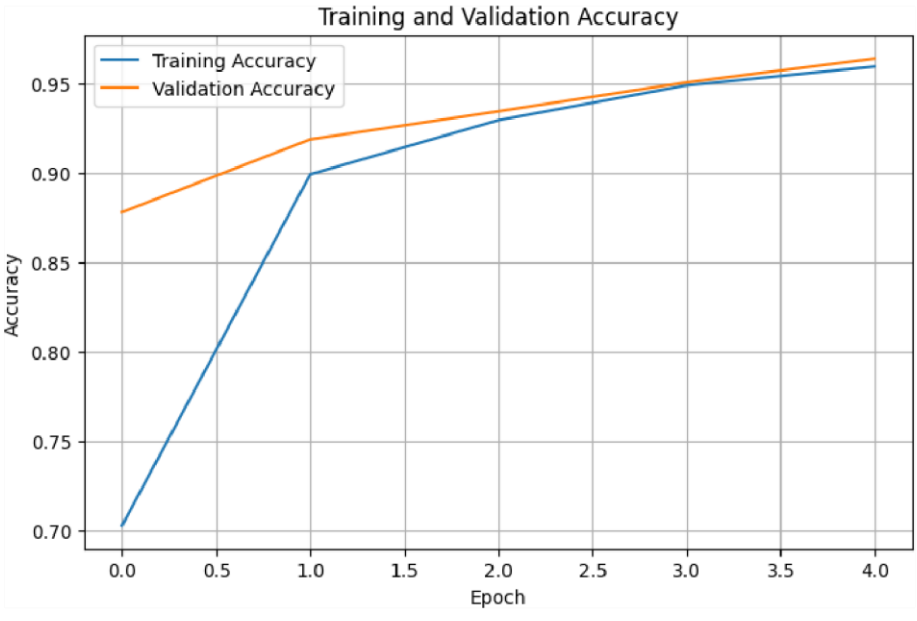
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()

plt.show()
```



```
# 16. LOSS GRAPH
# ----------------------------------------------------------------------------------------------------

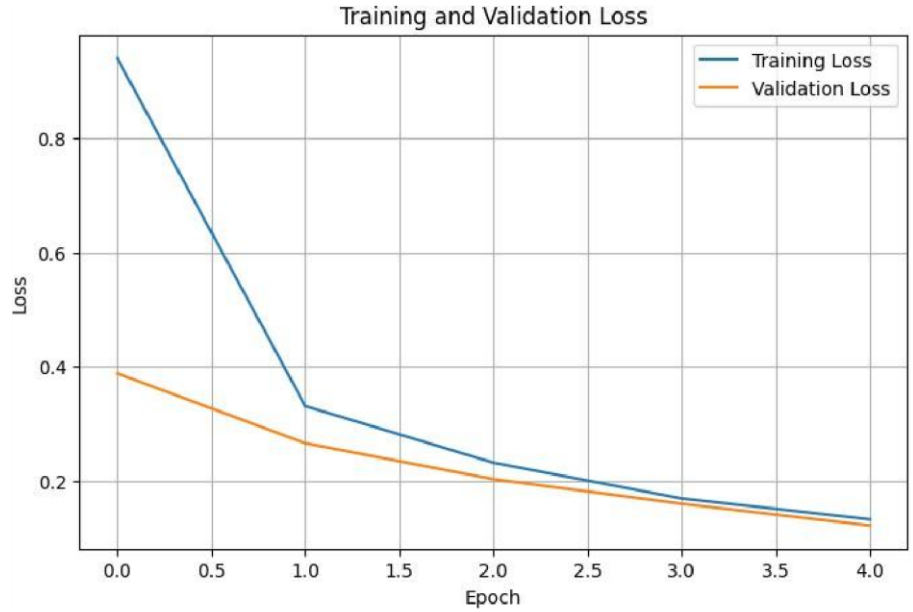
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid()

plt.show()
```




```
# 17. VALIDATION EVALUATION
# ----------------------------------------------------------------------------------------------------

print("\n============================================================")
print("MODEL EVALUATION")
print("============================================================")
print("PRIYANSHU SHARMA CU240250980")
val_loss, val_accuracy = model.evaluate(
    X_val,
    y_val,
    verbose=0
)

print("Validation Loss     :", round(val_loss, 4))
print(
    "Validation Accuracy :",
    round(val_accuracy * 100, 2),
    "%"
)
```

```
============================================================
MODEL EVALUATION
============================================================
PRIYANSHU SHARMA CU240250980
Validation Loss     : 0.1211
Validation Accuracy : 96.4 %
```